IF & CASE STATEMENT

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Reni Soelistijorini, B.Eng., MT

MATA KULIAH: Praktikum Rangkaian Logika 2

TANGGAL: Kamis, 28 Maret 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

CONDITIONAL dan SELECTED SIGNAL ASSIGNMENT

1. TUJUAN

- Mahasiswa dapat membuat progam dasar VHDL menggunakan *if dan case st atement.*
- Mahasiswa dapat membuat rangkaian logika sederhana menggunakan *if dan case statement.*
- Mahasiswa dapat membuat RTL dari rangkaian logika.
- Mahasiswa dapat membuat simulasi testbench dari rangkaian logika.
- Mahasiswa dapat membuat *Top Module* dari rangkaian logika.

2. ALAT DAN BAHAN

Alat dan Bahan untuk praktikum ini yaitu:

1. PC beserta software ISE Design Suite 13.1
2. Xilinx Spartan 3
3. Downloader JTAG USB
4. Power Supply 5V

3. PROSEDUR PERCOBAAN

A. Decoder 2 to 4 dengan if statement

1. Program untuk menampilkan decoder 2 to 4 dengan if statement adalah sebagai berikut :

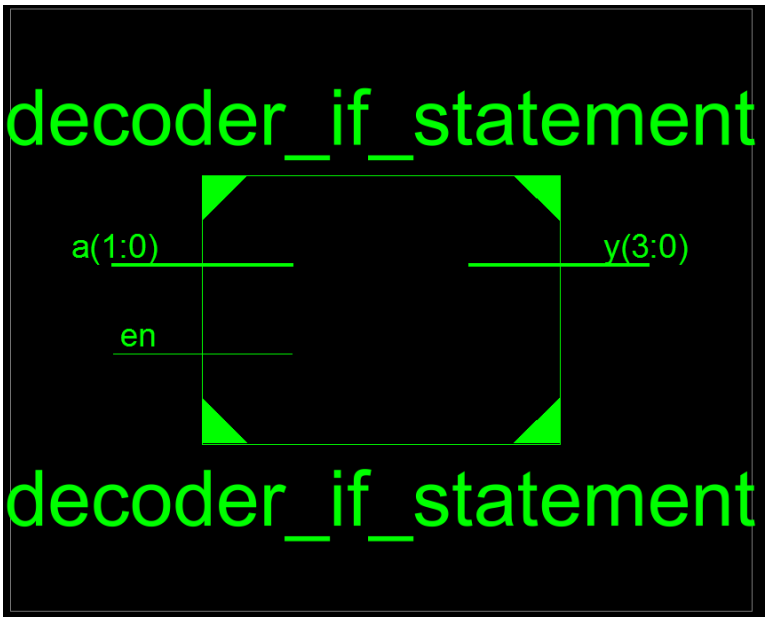
Konfigurasi pada file .vhd Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decoder_if_statement is
   Port ( a : in STD_LOGIC_VECTOR (1 downto 0);
          en : in STD_LOGIC;
          y : out STD_LOGIC_VECTOR (3 downto 0));
end decoder_if_statement;

architecture Behavioral of decoder_if_statement is
begin
   process(a, en)
   begin
      if en = '1' then
         case a is
            when "00" =>
               y <= "0001";
            when "01" =>
               y <= "0010";
            when "10" =>
```

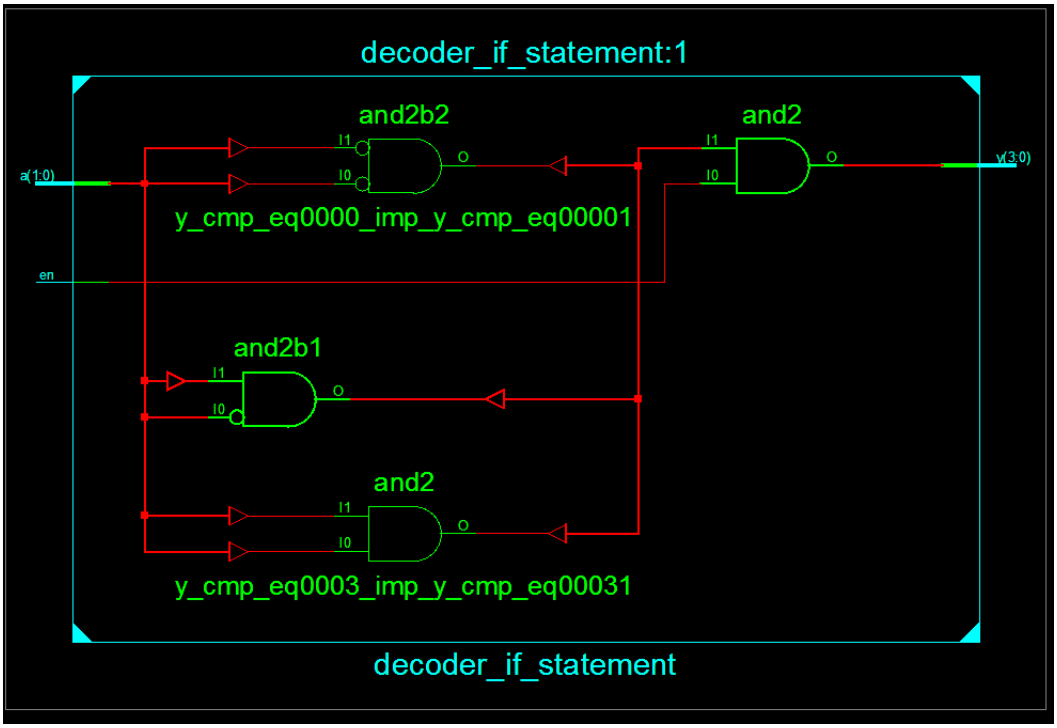
```
            y <= "0100";
          when "11" =>
            y <= "1000";
          when others =>
            y <= "0000";
        end case;
      else
        y <= "0000";
      end if;
    end process;
  end Behavioral;
```

2. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
3. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

4. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.



5. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.



6. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
7. Konfigurasikan file test bench (main_tb.vhd)

Konfigurasi pada file main_tb.vhd Simulation

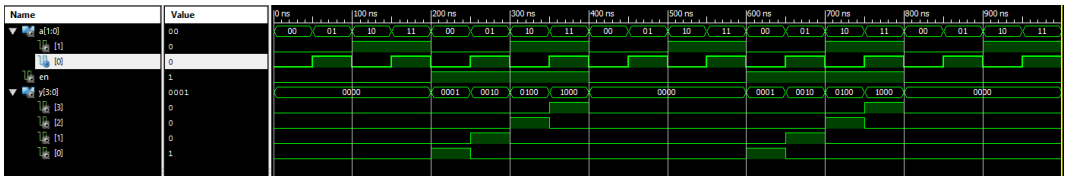
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decoder_if_statement_tb is
end decoder_if_statement_tb;

architecture Behavioral of decoder_if_statement_tb is
   -- Component declaration
   component decoder_if_statement
      Port ( a : in STD_LOGIC_VECTOR (1 downto 0);
             en : in STD_LOGIC;
             y : out STD_LOGIC_VECTOR (3 downto 0));
   end component;

   -- Signals declaration for stimulus
   signal a : std_logic_vector(1 downto 0);
   signal en : std_logic;
   signal y : std_logic_vector(3 downto 0);

begin
   -- Instantiate the decoder component
   uut: decoder_if_statement port map (a => a, en => en, y => y);

   -- Stimulus process
   stimulus_proc: process
   begin
      wait for 300 ns;
      en <= '0';
      wait for 250 ns;
      a <= "01";
      wait for 200 ns;
      a <= "10";
      wait for 150 ns;
      a <= "11";
```

```
        wait for 100 ns;
        en <= '1';
        a <= "00";
        wait;
      end process stimulus_proc;

    end Behavioral;
```

8. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

9. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk decoder 2 to 4 dengan if statement:



1 Time Diagram Decoder 2 to 4 dengan if Statement

B. Decoder 2 to 4 dengan case statement

1. Buatlah file baru untuk desain decoder 2 to 4 dengan case statement. Programnya adalah sebagai berikut :

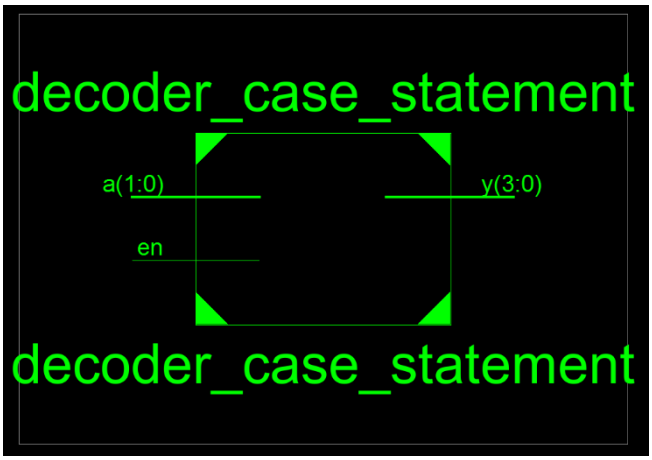
Konfigurasi pada file .vhd Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decoder_case_statement is
   Port ( a : in STD_LOGIC_VECTOR (1 downto 0);
          en : in STD_LOGIC;
          y : out STD_LOGIC_VECTOR (3 downto 0));
end decoder_case_statement;

architecture Behavioral of decoder_case_statement is
begin
   process(a, en)
   begin
      case a is
         when "00" =>
            if en = '1' then
               y <= "0001";
            else
               y <= "0000";
            end if;
         when "01" =>
            if en = '1' then
               y <= "0010";
            else
               y <= "0000";
            end if;
         when "10" =>
            if en = '1' then
               y <= "0100";
            else
               y <= "0000";
            end if;
         when "11" =>
            if en = '1' then
               y <= "1000";
            else
               y <= "0000";
```

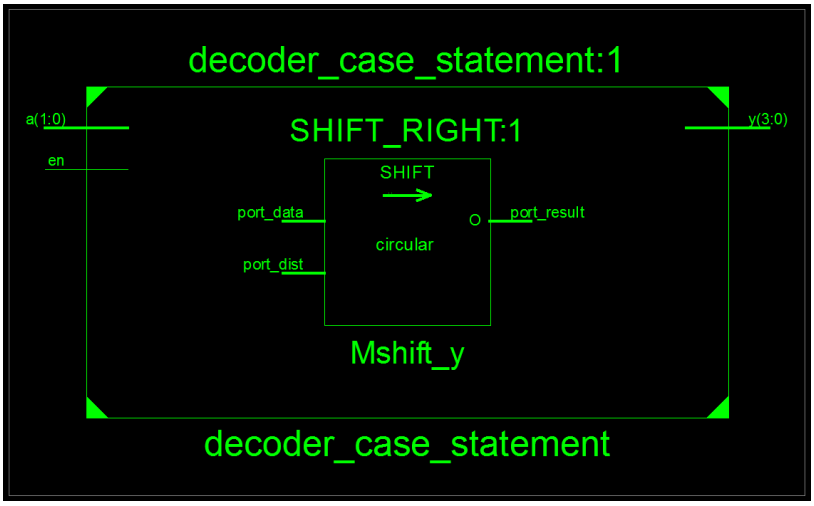
```
            end if;
          when others =>
            y <= "0000";
        end case;
      end process;
```

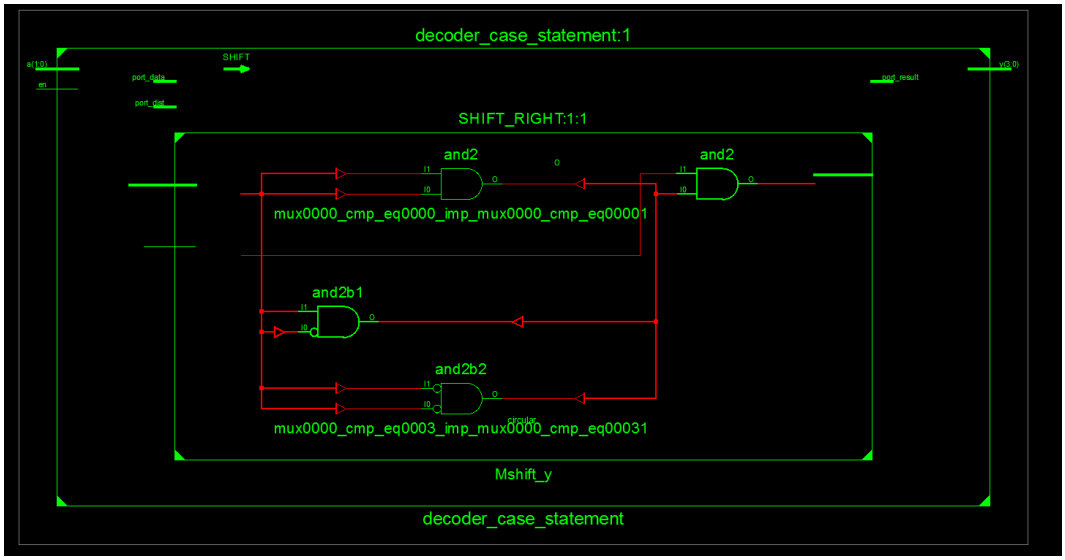
2. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
3. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

4. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.



5. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.





6. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
7. Konfigurasikan file test bench (main_tb.vhd)

Konfigurasi pada file main_tb.vhd Simulation

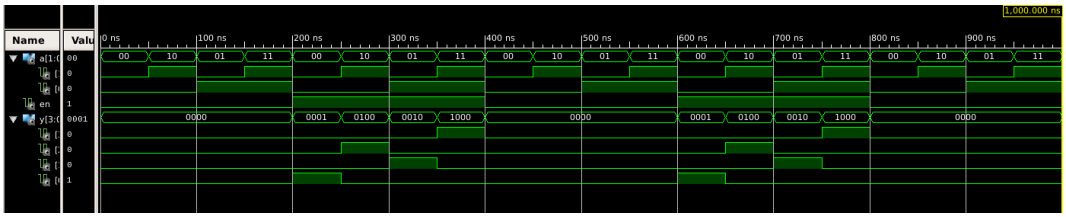
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY decoder_case_statement_tb IS
END decoder_case_statement_tb;
ARCHITECTURE behavior OF decoder_case_statement_tb IS
COMPONENT decoder_case_statement
PORT(
a : IN std_logic_vector(1 downto 0);
en : IN std_logic;
y : OUT std_logic_vector(3 downto 0)
);
END COMPONENT;
signal a : std_logic_vector(1 downto 0) := (others => '0');
signal en : std_logic := '0';
signal y : std_logic_vector(3 downto 0);
BEGIN
uut: decoder_case_statement PORT MAP (
a => a,
en => en,
y => y
);
stim_proc: process
begin
wait for 50 ns;
a(1) <= not a(1);
end process;
stim_proc1: process
begin
wait for 100 ns;
a(0) <= not a(0);
end process;
stim_prov2: process
begin
wait for 200 ns;
en <= not en;
end process;

END;
```

8. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run

untuk menampilkan simulasi.

9. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk decoder 2 to 4 dengan case statement:



2 Time Diagram Decoder 2 to 4 dengan Case Statement

C. Multiplexer 4x1

1. Program untuk menampilkan gerbang Multiplexer 4x1 adalah sebagai berikut :

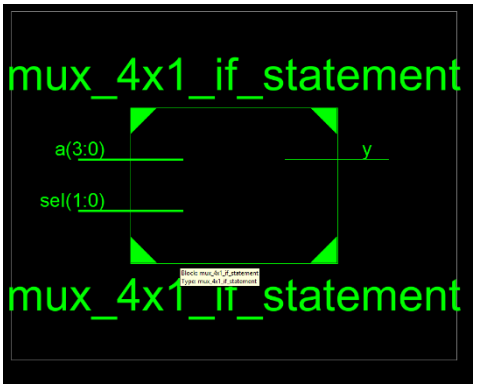
   Konfigurasi pada file .vhd Implementation

```
Multiplexer 4x1 dengan if statement:
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_4x1_if_statement is
   Port ( a : in STD_LOGIC_VECTOR (3 downto 0);
          sel : in STD_LOGIC_VECTOR (1 downto 0);
          y : out STD_LOGIC);
end mux_4x1_if_statement;

architecture Behavioral of mux_4x1_if_statement is
begin
   process(a, sel)
   begin
      if sel = "00" then
         y <= a(0);
      elsif sel = "01" then
         y <= a(1);
      elsif sel = "10" then
         y <= a(2);
      elsif sel = "11" then
         y <= a(3);
      else
         y <= '0'; -- Default value
      end if;
   end process;
end Behavioral;
```

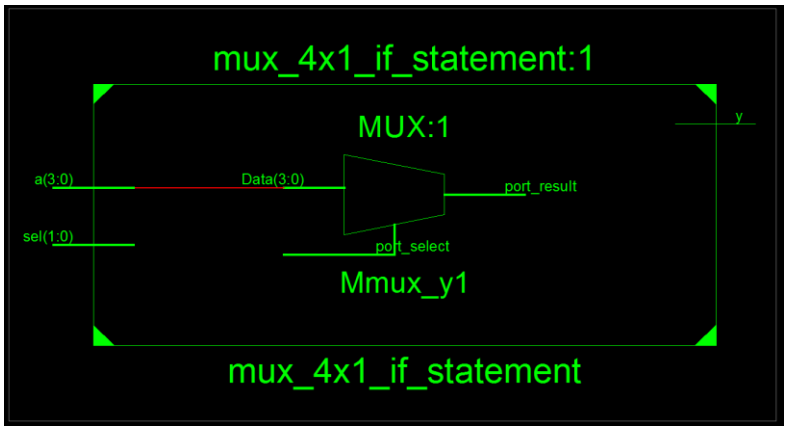
2. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.

3. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

4. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.



5. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.



6. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
7. Konfigurasikan file test bench (main_tb.vhd)

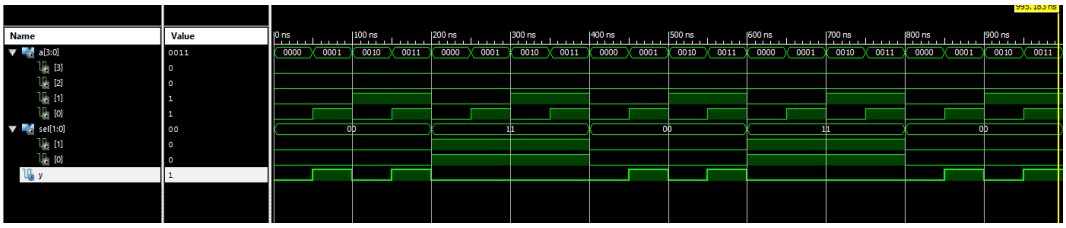
Konfigurasi pada file main_tb.vhd Simulation

```
library IEEE;
 use IEEE.STD_LOGIC_1164.ALL;

 entity mux_4x1_if_statement_tb is
 end mux_4x1_if_statement_tb;

 architecture Behavioral of mux_4x1_if_statement_tb is
    -- Component declaration
    component mux_4x1_if_statement
       Port ( a : in STD_LOGIC_VECTOR(3 downto 0);
              sel : in STD_LOGIC_VECTOR(1 downto 0);
              y : out STD_LOGIC);
    end component;

    -- Signals declaration for stimulus
    signal a : STD_LOGIC_VECTOR(3 downto 0);
    signal sel : STD_LOGIC_VECTOR(1 downto 0);
    signal y : STD_LOGIC;

 begin
    uut: mux_4x1_if_statement port map (a => a, sel => sel, y => y);

    stimulus_proc: process
    begin
       -- Test case 1: Select input 0, a = "1010"
       sel <= "00";
       a <= "1010";
       wait for 10 ns;

       -- Test case 2: Select input 1, a = "0110"
       sel <= "01";
       a <= "0110";
       wait for 10 ns;

       -- Test case 3: Select input 2, a = "1101"
       sel <= "10";
       a <= "1101";
       wait for 10 ns;

       -- Test case 4: Select input 3, a = "0011"
       sel <= "11";
       a <= "0011";
       wait for 10 ns;

       -- End simulation
       wait;
    end process stimulus_proc;

 end Behavioral;
```

8. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

9. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

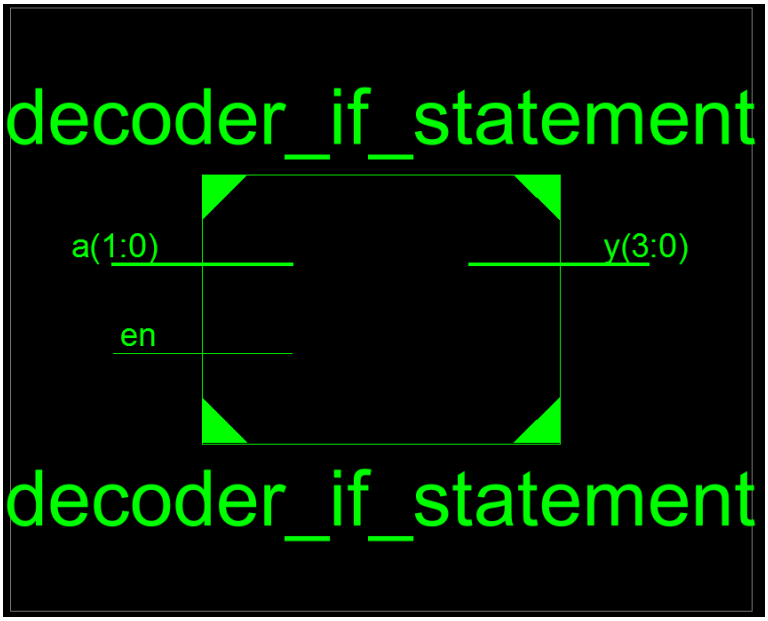
Hasil time diagram untuk Multiplexer 4x1

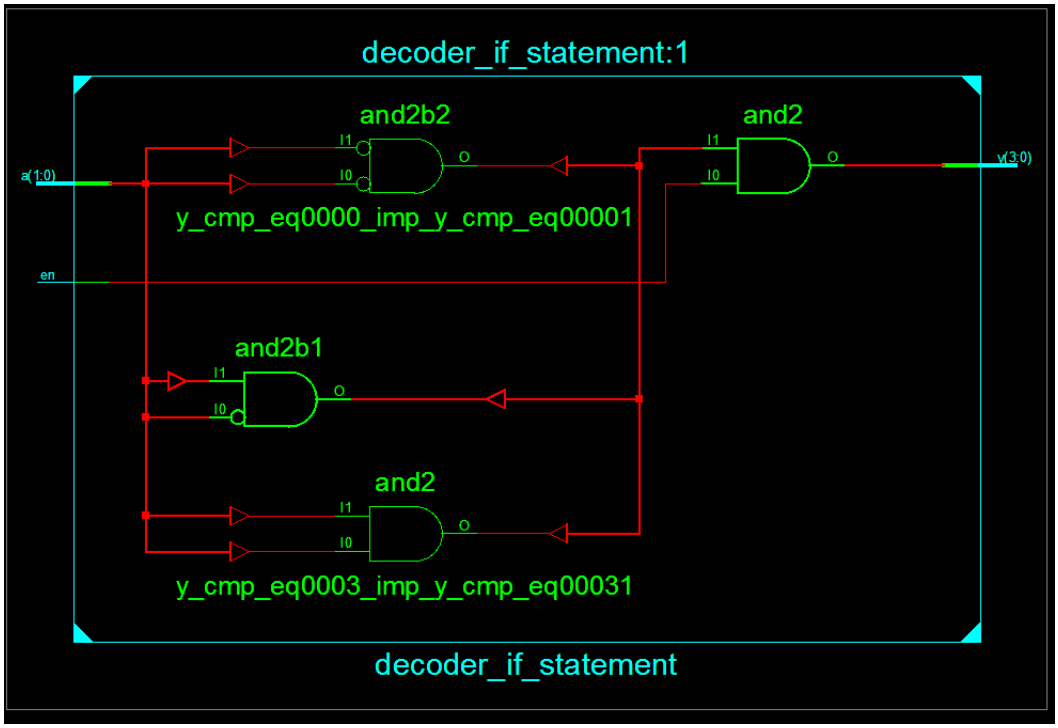


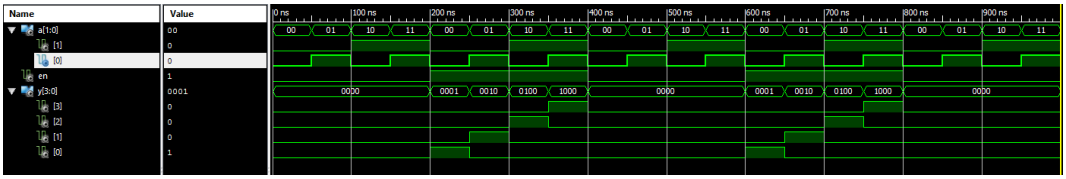
3 Time Diagram Multiplexee 4x1

D. HASIL DATA PERCOBAAN

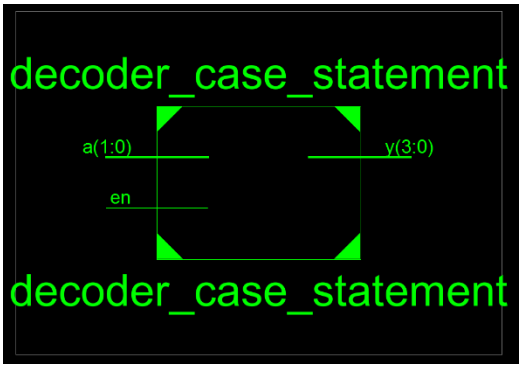
1. Hasil decoder 2 to 4 dengan if statement

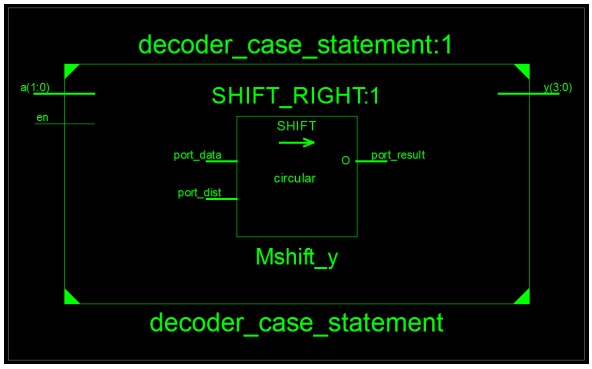


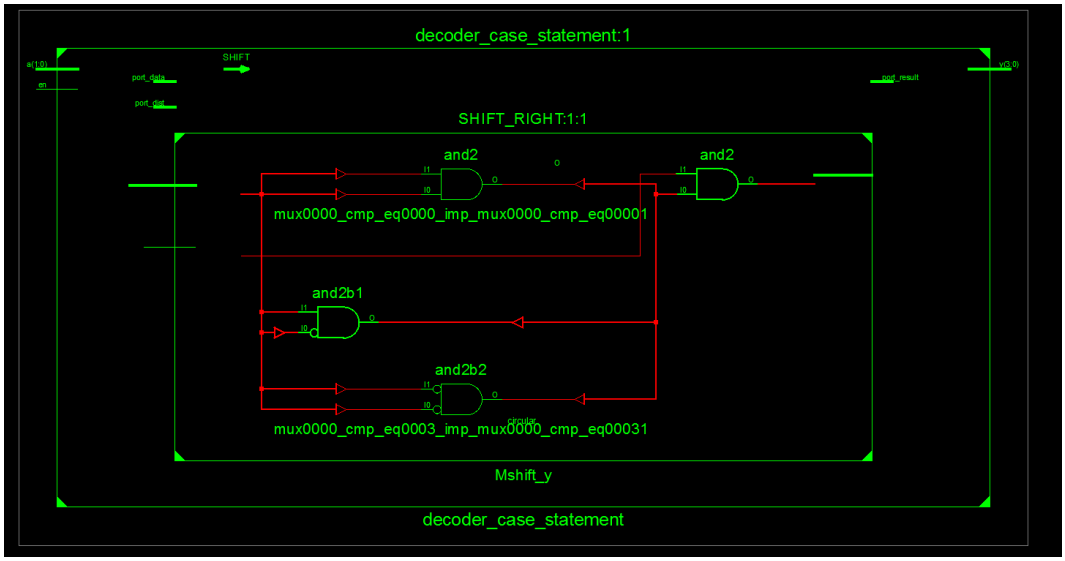


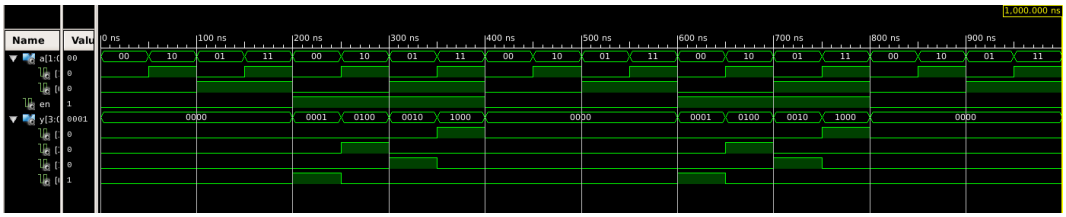


2. Hasil decoder 2 to 4 dengan case statement

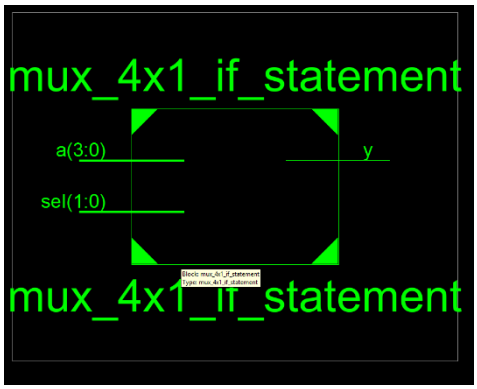


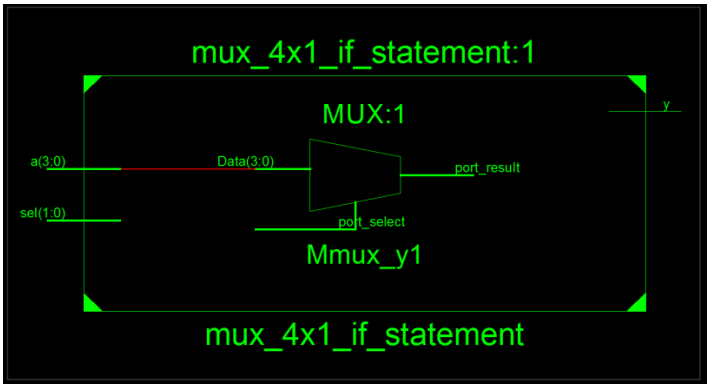


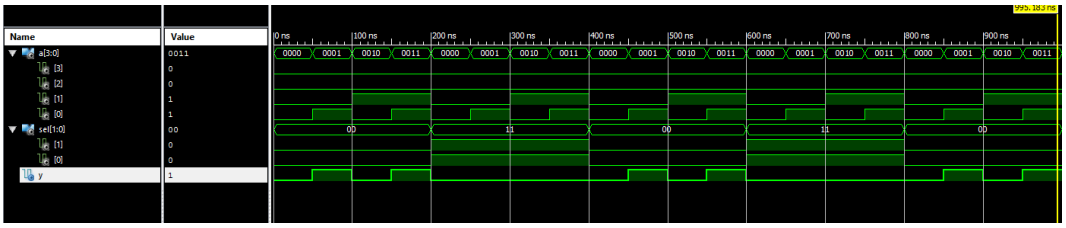




3. Hasil multiplexer 4x1





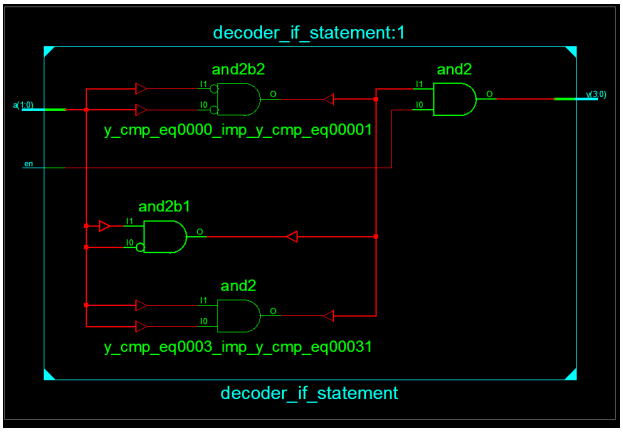


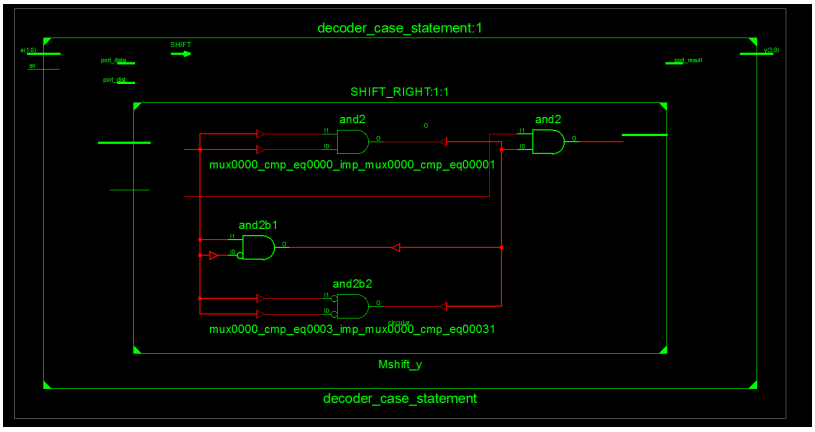
4. ANALISA

Dalam eksperimen menggunakan ISE Design Suite, berikut ini merupakan beberapa poin analisa:

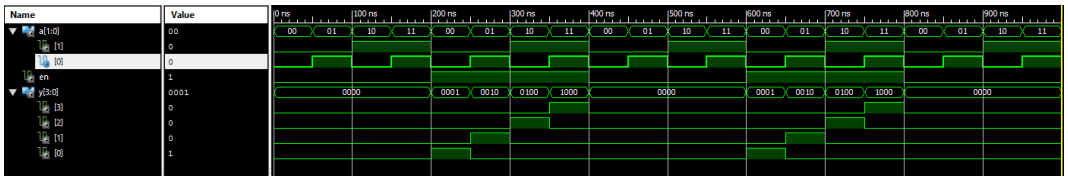
1. Decoder 2-to-4

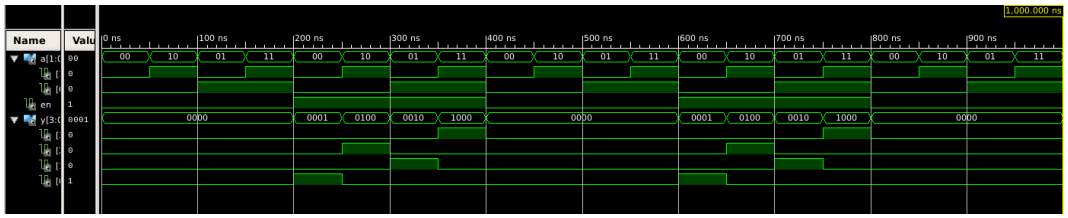
Sama seperti bab sebelumnya, decoder 2-to-4 adalah rangkaian digital yang mengubah input biner dua bit menjadi output yang merepresentasikan satu dari empat kemungkinan kombinasi output berdasarkan nilai input. Rangkaian ini umumnya digunakan dalam sistem yang memerlukan dekoding data biner menjadi representasi lain. Pada rangkaian Decoder 2-to-4, digunakan untuk mengimplementasikan logika yang menentukan output berdasarkan kondisi input yang diberikan.





Hasil simulasi rangkaian Decoder 2-to-4 menunjukkan pola sinyal input-output yang sesuai dengan logika yang diimplementasikan. Pada time diagram, terlihat perubahan output sesuai dengan perubahan input yang diberikan.





Analisis pola sinyal input-output pada time diagram menunjukkan bahwa rangkaian bekerja sesuai dengan spesifikasi yang diinginkan. Setiap perubahan pada input langsung tercermin pada output yang dihasilkan berdasarkan kondisi yang ditetapkan. Rangkaian Decoder 2-to-4 dapat diimplementasikan dengan logika yang bersih dan mudah dipahami. Hal ini memudahkan dalam memahami dan mengelola rangkaian digital yang kompleks.

Kelebihan dan kekurangan decoder 2to4 dengan If statement

- Kelebihan: Implementasi dengan if statement relatif mudah dipahami dan dapat diimplementasikan dengan cepat.
- Kekurangan: Kode bisa menjadi lebih panjang jika terdapat banyak kondisi yang harus diperiksa dengan if statement.

Kelebihan dan kekurangan decoder 2to4 dengan Case statement

- Kelebihan: Case statement dapat menyederhanakan kode jika terdapat banyak kondisi yang harus diperiksa.
- Kekurangan: Dalam beberapa kasus, if statement mungkin lebih mudah dipahami daripada case statement, terutama jika terdapat kondisi yang kompleks.

Kita juga dapat melihat pada table kebenaran berdasarkan logika yang diimplmentasikan dalam kode VHDL di program ini.

en	a1	a0	y3	y2	y1	y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Keterangan:
- a1, a0 adalah input rangkaian.
- en adalah input enable.
- y3, y2, y1, y0 adalah output rangkaian.

Dalam tabel kebenaran di atas, X menunjukkan bahwa input tidak terdefinisi. Output y3-y0 sesuai dengan logika rangkaian Decoder 2 to 4 yang ada. Saat enable (en) aktif (1), output sesuai dengan input a1 dan a0 yang diberikan. Jika enable (en) tidak aktif (0), semua output y3-y0 akan bernilai 0.

Hasil percobaan ini mengkonfirmasi bahwa kedua implementasi berhasil menghasilkan pola sinyal input-output yang sesuai dengan spesifikasi Decoder 2-to-4. Pilihan antara if statement dan case statement bergantung pada preferensi pengembang dan kompleksitas logika yang diinginkan. If statement lebih mudah dipahami dalam beberapa kasus sederhana, sementara case statement dapat menyederhanakan kode jika terdapat banyak kondisi yang harus diperiksa.

2. Multiplexer 4x1

Multiplexer 4x1 adalah sebuah rangkaian digital yang memiliki empat input (a) dan dua input selektor (sel). Implementasi Multiplexer 4x1 menggunakan if statement dalam VHDL, seperti yang ditunjukkan dalam kode yang diberikan, memungkinkan kita untuk memilih salah satu input sebagai output berdasarkan nilai dari selektor.

Dalam implementasi ini, output (y) dari Multiplexer 4x1 akan ditentukan oleh nilai selektor (sel). Jika nilai selektor adalah "00", maka output (y) akan menjadi nilai dari input pertama (a(0)). Jika selektor adalah "01", output (y) akan mengambil nilai dari input kedua (a(1)). Demikian pula, jika selektor adalah "10" atau "11", output (y) akan mengambil nilai dari input ketiga (a(2)) atau input keempat (a(3)), sesuai urutan.

Analisa pada main.vhdl dan test bench:
Pada entitas mux_4x1_if_statement, terdapat tiga sinyal yang dideklarasikan: input a dengan ukuran STD_LOGIC_VECTOR 4 bit (a(3) hingga a(0)), input selektor sel dengan ukuran STD_LOGIC_VECTOR 2 bit (sel(1) dan sel(0)), dan output y dengan tipe STD_LOGIC.

Selanjutnya, dalam arsitektur Behavioral dari multiplexer, terdapat proses yang dipicu oleh perubahan nilai sinyal a dan sel. Pada proses ini, dilakukan pengecekan kondisi if-else berdasarkan nilai selektor (sel) yang diberikan.

- Jika nilai selektor adalah "00", maka output y akan diatur sesuai dengan nilai dari input pertama a(0).
- Jika nilai selektor adalah "01", output y akan mengambil nilai dari input kedua a(1).
- Jika nilai selektor adalah "10", output y akan diatur sesuai dengan nilai dari input ketiga a(2).
- Jika nilai selektor adalah "11", output y akan mengambil nilai dari input keempat a(3).
- Jika nilai selektor tidak sesuai dengan kondisi di atas (misalnya, tidak bernilai "00", "01", "10", atau "11"), maka output y akan diatur menjadi '0' sebagai nilai default.

Hal ini sesuai juga dengan table kebenarannya:

S(1)	S(0)	A(3)	A(2)	A(1)	A(0)	Output Y
0	0	0	x	x	x	0
0	0	1	x	x	X	1
0	1	x	0	x	X	0
0	1	x	1	x	X	1
1	0	x	x	0	X	0
1	0	x	x	1	X	1
1	1	X	x	X	0	0
1	1	x	X	x	1	1

Selain itu, untuk menguji fungsionalitas multiplexer, dibuat juga entitas mux_4x1_if_statement_tb yang merupakan testbench untuk simulasi. Pada testbench ini, sinyal-sinyal input dan selektor diberikan nilai secara bergantian untuk menguji respons multiplexer terhadap berbagai kombinasi input dan selektor yang mungkin terjadi. Analisis keseluruhan dari implementasi ini adalah bahwa multiplexer bekerja sesuai dengan logika yang diharapkan, di mana output akan bergantung pada nilai selektor yang dipilih

Kelebihan dari implementasi ini adalah kejelasan dan kemudahan dalam pemahaman logika yang diimplementasikan. Namun, jika terdapat banyak kondisi seleksi, kode dapat menjadi panjang dan memerlukan banyak pernyataan if-else, yang dapat mempersulit pemeliharaan dan pengembangan kode di masa mendatang. Dalam kasus penggunaan Multiplexer 4x1 dengan if statement ini, cocok digunakan untuk situasi yang relatif sederhana dengan jumlah input yang terbatas dan kondisi seleksi yang tidak terlalu kompleks..

5. KESIMPULAN

Kesimpulan dari perbandingan penggunaan if dan case dalam VHDL adalah bahwa keduanya memiliki tujuan dan penggunaan yang berbeda. If statement digunakan untuk memeriksa kondisi boolean dengan dua pilihan, yaitu true atau false, serta dapat digunakan untuk kondisi yang kompleks dengan beberapa variabel sekaligus. Prioritas kondisi awal dalam if statement lebih tinggi dibandingkan dengan kondisi selanjutnya. Sementara itu, case statement digunakan untuk memeriksa nilai ekspresi dan memilih satu dari banyak kasus yang berbeda. Case

statement hanya cocok untuk kondisi yang sederhana dan spesifik, dan semua when dalam case statement memiliki prioritas yang sama.

Pada implementasi VHDL seperti program decoder 2 to 4 dan multiplexer 4x1, if statement digunakan untuk logika keputusan yang lebih berurutan, memeriksa nilai secara bertahap dan memberikan output sesuai dengan kondisi yang terpenuhi. Sementara itu, case statement lebih cocok digunakan untuk logika keputusan yang memiliki beberapa kasus yang berbeda dan tidak terlalu kompleks. Dalam kedua kasus tersebut, penggunaan if dan case statement dapat memberikan solusi yang efektif sesuai dengan kebutuhan desain.

Dengan pengujian menggunakan testbench pada kode VHDL yang diimplementasikan, fungsi dan perilaku dari masing-masing komponen seperti decoder, multiplexer, dan rangkaian logika kombinasional dapat diverifikasi dengan baik. Pengujian ini memastikan bahwa komponen-komponen tersebut beroperasi sesuai dengan spesifikasi yang diinginkan, serta memberikan hasil output yang diharapkan berdasarkan kondisi dan ekspresi yang diberikan dalam kode VHDL.